An

Industrial Oriented Mini Project

On

IMAGE-BASED PADDY DISEASE RECOGNITION USING RESNET101 DEEP NETWORK

(Submitted in partial fulfilment of the requirements for the award of the Degree)

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

By

Kommula Rajesh (227R1A1299)

Under the Guidance of

Mr. Md. Sajid Pasha

(Associate Professor)

DEPARTMENT OF INFORMATION TECHNOLOGY

CMR TECHNICAL CAMPUS

UGC AUTONOMOUS

(Accredited by NBA& NAAC with 'A' Grade Approved By AICTE, NewDelhi and JNTU Hyderabad, Kandlakoya(V), Medchal Road, Hyderabad, Telangana-501401)

2025-2026

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled **"Image-Based Paddy Disease Recognition using ResNet101 Deep Network"** has being submitted by **K.Rajesh (227R1A1299),**in partial fulfilment of the requirements for the award of degree of B. Tech in Information Technology of Jawaharlal Nehru Technological University Hyderabad, is a record for the bonafide work carried out by them under our guidance and supervision during the academic year **2024-2025**.

The results embodied in this project have not been submitted by any other university or institute for the award of degree or diploma.

Md. Sajid Pasha
ASSOCIATE PROFESSOR **HEAD OF THE DEPARTMENT**
INTERNAL GUIDE

Dr. A. Raji Reddy **EXTERNAL EXAMINER**
DIRECTOR

Submitted for viva voice Examination held on_____

ACKNOWLEDGEMENT

Apart from the efforts of us, the success of any work depends largely on the encouragement and guidelines of many others.We take this opportunity to express our gratitude to the people who have been instrumental in the successful completion of this project.

We take this opportunity to express our profound gratitude and deep regard to our guide faculty, **Mr.Md.Sajid Pasha**,Assistant Professor for her exemplary guidance, monitoring and constant encouragement throughout the course of this Project Work.

We take this opportunity to express our profound gratitude to our PRC coordinators, **Ms.G.Menaka,** Assistant Professor and **Mrs. Arfa Mahwish**, Assistant Professor. The blessing, help and guidance given by their time to time shall carry us a long way in the journey ahead that, we are about to embark upon.

We are also thankful to **Dr. B. Kavitha Rani,** Professor & Head of the Department for providing excellent infrastructure and a conducive atmosphere for contributing to the successful completion of project.

We would like to express our sincere gratitude **to Dr. M. Ahmed Ali Baig**, Dean Administration, **Dr.DTV Dharmajee Rao**, Dean Academics, **Dr.Ashutosh Saxena**, Dean R&D for encouragement throughout the course of this project.

We obliged to our Director **Dr. A. RajiReddy** for his cooperation throughout the completion of this project.

We wish to express our sincere gratitude to the Management of **CMR Technical Campus**, Hyderabad, **Sri C. Gopal Reddy**, Honorable Chairman, **Smt C. Vasantha Latha**, Honorable Secretary, **Sri C. Abhinav Reddy**, Honorable Chief Executive Officer.

The guidance and support received from all the members of **CMR TECHNICAL CAMPUS** who contributed and who are contributing to this project, was vital for the success of the project. We are grateful for their constant support and help.

Finally, We would like to take this opportunity to thank our family for their constant encouragement without which this assignment would not be possible. We sincerely acknowledge and thank all those who provide support, both directly and indirectly in completion of this project.

K.Rajesh (227R1A1299)

ABSTRACT

Paddy(rice) is a staple food crop consumed by over half the world's population. However, the yield and quality of rice are frequently threatened by various leaf diseases such as bacterial blight, brown spot, and leaf blast. Early and accurate diagnosis of these diseases is crucial for effective management and to minimize crop losses. Traditional methods rely heavily on manual inspection, which is time-consuming, labor-intensive, and prone to human error.This project proposes an automated, image-based paddy disease recognition system using ResNet101, a deep convolutional neural network (CNN) known for its powerful feature extraction and deep architecture. The model is trained on a labeled dataset of paddy leaf images, representing both healthy and diseased conditions. The ResNet101 model, with its residual learning and 101 layers, enables high accuracy in detecting subtle variations in disease patterns. The system takes a paddy leaf image as input and classifies it into predefined disease categories with high precision.

LIST OF FIGURES

FIGURE NO	NAME OF THE FIGURE	PAGE NO.
Figure 5.1	Proposed Architecture	11
Figure 5.2.1	Use Case Diagram	13
Figure 5.2.2	Class Diagram	14
Figure 5.2.3	Sequence Diagram	15
Figure 5.2.4	Flowchart Diagram	16

LIST OF SCREENSHOTS

SCREENSHOT NO.	SCREENSHOT NAME	PAGE NO.
Screenshot 7.1	Welcome Screen of Image-Based Paddy Disease Recognition Using ResNet101 Deep Network	27
Screenshot 7.2	Uploading Dataset	27
Screenshot 7.3	Preprocessing Dataset	28
Screenshot 7.4	Running ResNet Algorithm	28
Screenshot 7.5	Comparison Graph	29
Screenshot 7.6	Predict Disease from Test Image	29
Screenshot 7.7	Predicting Disease as Healthy	30
Screenshot 7.8	Predicting Disease as Leaf Blast	30

INDEX

TITLE	PAGE NO.
ABSTRACT	**i**
LIST OF FIGURES	**ii**
LIST OF SCREENSHOTS	**iii**
1. INTRODUCTION	**1**
1.1 PROJECT SCOPE	2
1.2 PROJECT PURPOSE	2
1.3 PROJECT FEATURES	2
2. SYSTEM ANALYSIS	**3**
2.1 LITERATURE SURVEY	4
2.2 PROJECT STATEMENT	4
2.3 EXISTING SYSTEM	4
2.4 DISADVANTAGES OF EXISTING SYSTEM	4
2.5 PROPOSED SYSTEM	4
2.6 ADVANTAGES OF PROPOSED SYSTEM	5
3. SYSTEM STUDY	**6**
3.1 FEASIBILITY STUDY	7
3.1.1 ECONOMICAL FEASIBILITY	7
3.1.2 TECHNICAL FEASIBILITY	7
3.1.3 OPERATIONAL FEASIBILITY	7
4. SYSTEM REQUIREMENTS	**8**
4.1 SOFTWARE REQUIREMENTS	9
4.2 HARDWARE REQUIREMENTS	9

5. SYSTEM DESIGN **10**

5.1 INTRODUCTION 11

5.2 SYSTEM ARCHITECTURE 11

5.3 UNIFIED MODELING LANGUAGE 12

5.3.1 USECASE DIAGRAM 13

5.3.2 CLASS DIAGRAM 14

5.3.3 SEQUENCE DIAGRAM 15

5.3.4 FLOW CHART DIAGRAM 16

6. IMPLEMENTATION **17**

6.1 MODULES 18

6.2 SOURCE CODE 19

7. SCREENSHOTS **26**

8. TESTING **31**

8.1 INTRODUCTION TO TESTING 32

8.2 TYPES OF TESTING 32

8.2.1 UNIT TESTING 32

8.2.2 INTEGRATION TESTING 32

8.2.3 FUNCTIONALITY TESTING 33

8.2.4 SYSTEM TESTING 33

8.2.5 BLACKBOX TESTING 33

8.2.6 WHITEBOX TESTING 33

8.2.7 PERFORMANCE TESTING 34

8.2.8 ACCEPTANCE TESTING 34

8.3 TESTCASES 34

9. CONCLUSION & FUTURE SCOPE **35**

9.1 PROJECT CONCLUSION 36

9.2 FUTURE SCOPE 36

10. BIBLIOGRAPHY **37**

10.1 REFERENCES 38

IMAGE-BASED PADDY DISEASE RECOGNITION USING RESNET101 DEEP NETWORK

1.INTRODUCTION

1. INTRODUCTION

1.1 PROJECT SCOPE

The project aims to identify and classify common rice leaf diseases such as bacterial blight, brown spot, and leaf blast from digital images with high accuracy. The project involves the collection and preprocessing of a labeled dataset of paddy leaf images, training the ResNet101 model to extract deep features, and evaluating its performance through various metrics.This project lays the foundation for scalable, technology-driven solutions in precision agriculture, aiming to support farmers with early disease diagnosis and better crop management.

1.2 PROJECT PURPOSE

The main purpose of this project is to:

1. Develop a deep learning-based model to accurately detect and classify diseases in paddy leaves from images.
2. Help farmers and agricultural officers in early detection of paddy diseases using a non-invasive and automated method.
3. Reduce the economic impact of crop diseases by enabling faster diagnosis and timely treatment.
4. Provide a foundation for future integration with mobile or drone-based agricultural monitoring systems.

1.3 PROJECT FEATURES

1. Automated Disease Detection
2. Image-Based Analysis
3. Multi-Class Classification
4. Performance Evaluation

2.SYSTEM ANALYSIS

1. SYSTEM ANALYSIS

1.1 LITERATURE SURVEY

The rapid advancement of deep learning and computer vision techniques has revolutionized agricultural practices, particularly in the domain of automated crop disease detection. Numerous studies have explored various Convolutional Neural Networks (CNNs) and machine learning algorithms to identify paddy leaf diseases accurately and efficiently.

2.2 PROJECT STATEMENT

To design and develop an automated image-based system for detecting and classifying paddy leaf diseases using the ResNet101 deep convolutional neural network. The system aims to improve the speed, accuracy, and reliability of disease identification by analyzing visual symptoms on paddy leaves. By leveraging the power of deep learning and transfer learning, the project intends to support early diagnosis and effective disease management in rice cultivation, ultimately enhancing crop yield and reducing manual dependency in agricultural monitoring.

2.3 EXISTING SYSTEM

In current agricultural practices, the identification of paddy leaf diseases is predominantly carried out through **manual inspection** by farmers or agricultural experts. The general process of using traditional image recognition processing technology to identify plant diseases is shown in Fig. 1. Dubey and Jalal [3] used the K-means clustering method to segment the lesions regions, and combined the global color histogram (GCH) color coherence vector (CCV) local binary pattern (LBP), and completed local binary pat- tern (CLBP) was used to extract the color and texture features of apple spots, and three kinds of apple diseases were detected and identified based on improved support vector machine (SVM), and the classification accuracy reached 93%.

2.4 DISADVANTAGES OF EXISTING SYSTEM

1. Less accuracy
2. Low Efficiency
3. Manual and Time-Consuming

2.5 PROPOSED SYSTEM

The proposed system aims to develop an automated, deep learning-based approach for identifying

and classifying diseases in paddy leaves using the ResNet101 convolutional neural network architecture. This system takes a digital image of a paddy leaf as input and classifies it into one of several predefined disease categories or as healthy.

1. Image Acquisition
2. Image Preprocessing
3. Model Architecture (ResNet101)
4. Training & Validation
5. Disease Classification
6. Prediction Output

2.6 ADVANTAGES OF PROPOSED SYSTEM

1. High accuracy
2. High efficiency

3.SYSTEM STUDY

2. SYSTEM STUDY

2.1 FEASIBILITY STUDY

A feasibility study helps determine whether the proposed system is viable in terms of implementation, affordability, practicality, and societal acceptance.

Three key considerations involved in the feasibility analysis are:

2.1.1 ECONOMICAL FEASIBILITY

The economic feasibility assesses whether the project is cost-effective and affordable within available resources.The model is trained using open-source tools like Python, TensorFlow/Keras, and Google Colab, which are free to use.Image datasets are publicly available or can be collected with minimal cost using mobile phones.No expensive hardware is required during development; a standard computer or cloud resources are sufficient.

2.1.2 TECHNICAL FEASIBILITY

The Technical feasibility evaluates whether the system can be built with the current technology and knowledge available.The use of ResNet101, a well-established deep CNN architecture, ensures high accuracy and robustness in image classification tasks.Tools such as Python, OpenCV, TensorFlow, and Keras support rapid model development and deployment.The system architecture allows easy extension to new diseases or crops by retraining with new data.The project can run on standard systems during development and can later be deployed in cloud or mobile environments.

2.1.3 OPERATIONAL FEASIBILITY

This assesses how acceptable and useful the system is to end users — in this case, farmers and agricultural professionals.Farmers suffer significant losses due to late or incorrect disease identification; this system provides an accessible solution.With the rise of smartphone usage in rural areas, the system can be easily made available as a mobile app.It can increase awareness, reduce dependency on experts, and empower farmers to take timely action.

4.SYSTEM REQUIREMENTS

3. SYSTEM REQUIREMENTS

3.1 SOFTWARE REQUIREMENTS

1. Front end Technologies : HTML,CSS
2. Back end Technologies : Python
3. Framework : Django

4.2 HARDWARE REQUIREMENTS

1. Processor : intel i5
2. Hard Disk : 1TB
3. RAM : 4GB

5.SYSTEM DESIGN

4. SYSTEM DESIGN

4.1 INTRODUCTION

System design is the blueprint of how the paddy disease recognition system works. It outlines the structure and flow of components such as image input, preprocessing, disease detection using the ResNet101 deep learning model, and result output.

5.2 SYSTEM ARCHITECTURE

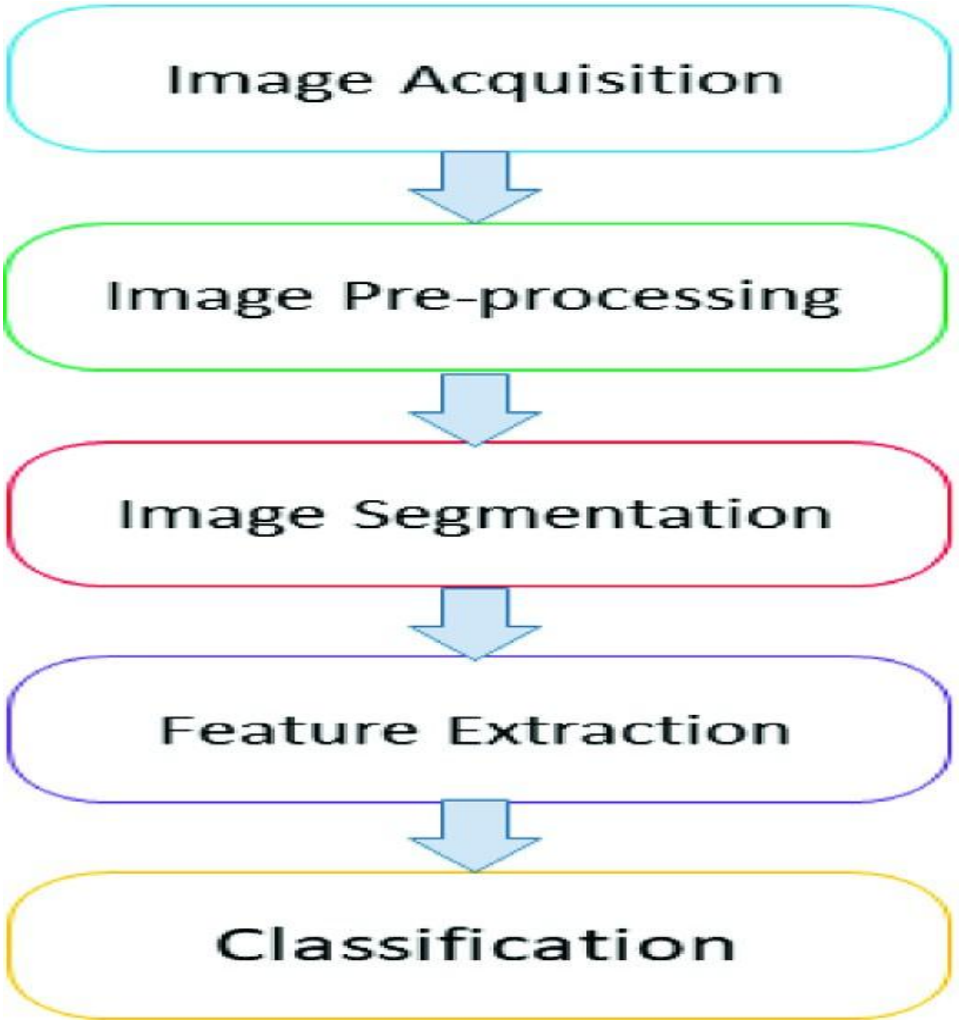


FIGURE 5.2 PROPOSED ARCHITECTURE

5.3 UNIFIED MODELING LANGUAGE(UML)

UML stands for Unified Modeling Language. It is an institutionalized, general-purpose modeling language in the field of object-oriented software engineering. The standard is maintained and was created by the Object Management Group (OMG).The goal of UML is to become a standard language for creating models of object-oriented computer software. In its current form, UML consists of two major components: a meta-model and a notation. Later on, various methods or systems can also be added to or associated with UML. The Unified Modeling Language is a widely used language for specifying, visualizing, constructing, and documenting the artifacts of software systems, as well as for business modeling and other non-software systems.UML represents a collection of best engineering practices that have proven successful in modeling large and complex systems. It is an essential part of developing object-oriented software and plays a significant role in the software development process. UML primarily uses graphical notations to express the design of software projects.

GOALS OF UML:

The primary goals in the design of UML are as follows:

1. To provide users with a ready-to-use, expressive visual modeling language so that they can develop and exchange meaningful models.
2. To provide extensibility and specialization mechanisms to extend the core concepts.
3. To be independent of particular programming languages and development processes.
4. To provide a formal basis for understanding the modeling language.

.

5.3.1 USECASE DIAGRAM

A use case diagram in the Unified Modelling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those usecases. The main purpose of a usecase diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

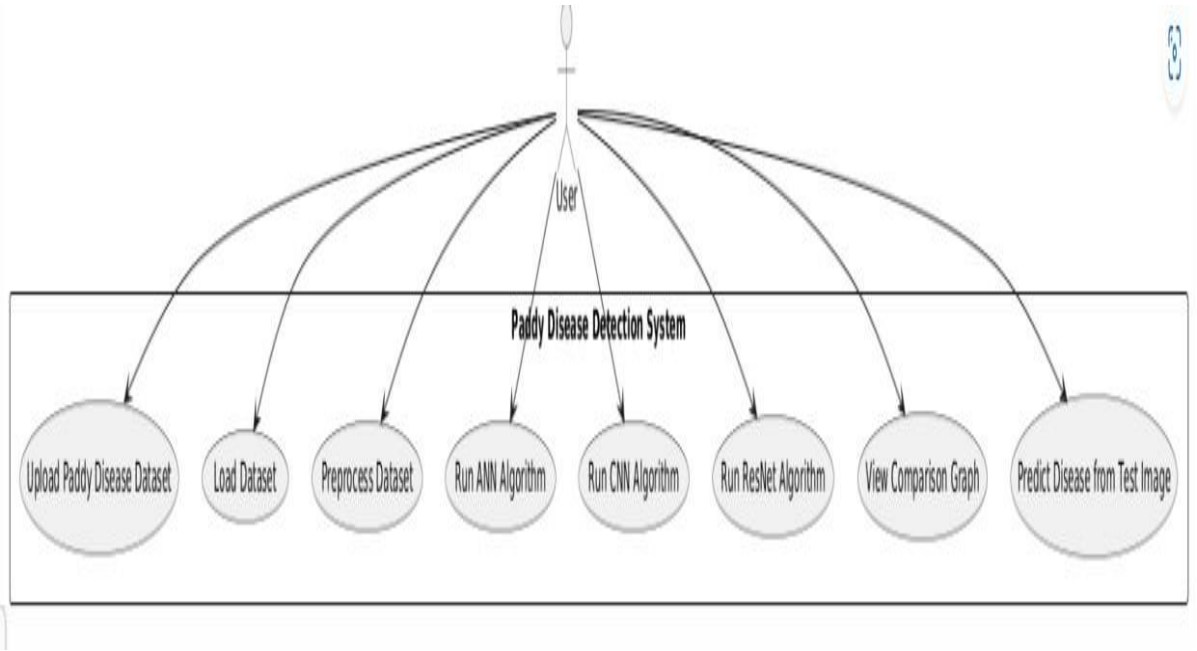


FIGURE 5.3.1 USE CASEDIAGRAM

5.3.2 CLASS DIAGRAM

In software engineering, a class diagram in the Unified Modelling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations(or methods), and the relationships among the classes. It explains which class contains information.

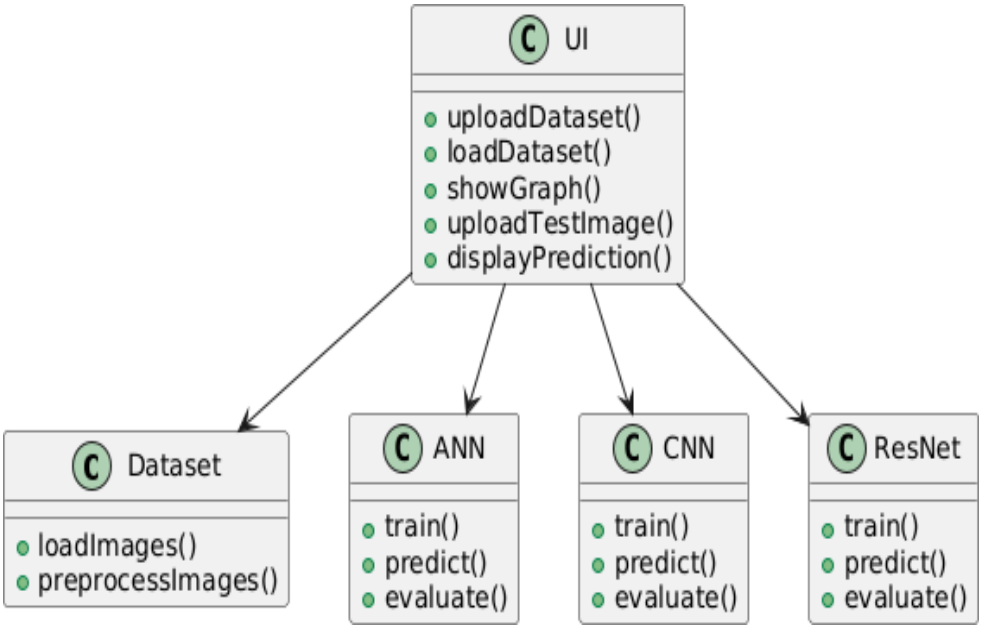


FIGURE 5.3.2 CLASS DIAGRAM

5.3.3 SEQUENCE DIAGRAM

A sequence diagram in Unified Modelling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

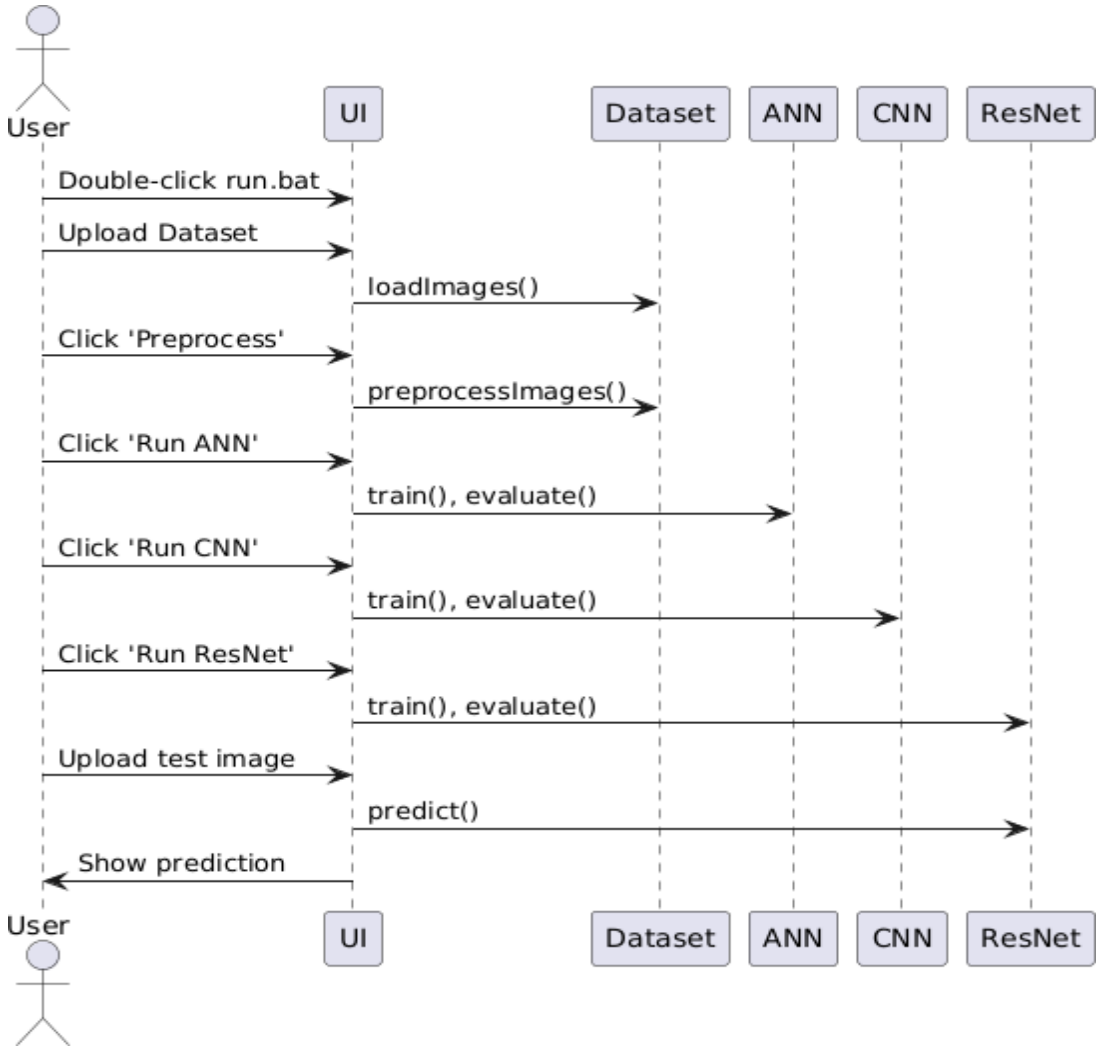


FIGURE 5.3.3 SEQUENCE DIAGRAM

5.3.4 FLOWCHART DIAGRAM:

A flow chart diagram is a visual tool that shows the steps in a processor work-flow. It uses shapes like arrows and rectangles to represent different actions and decisions. Flowcharts help simplify complex tasks and make it easier to understand how things work. They are useful for planning and identifying areas for improvement. Overall, flowcharts help communicate processes clearly to others.

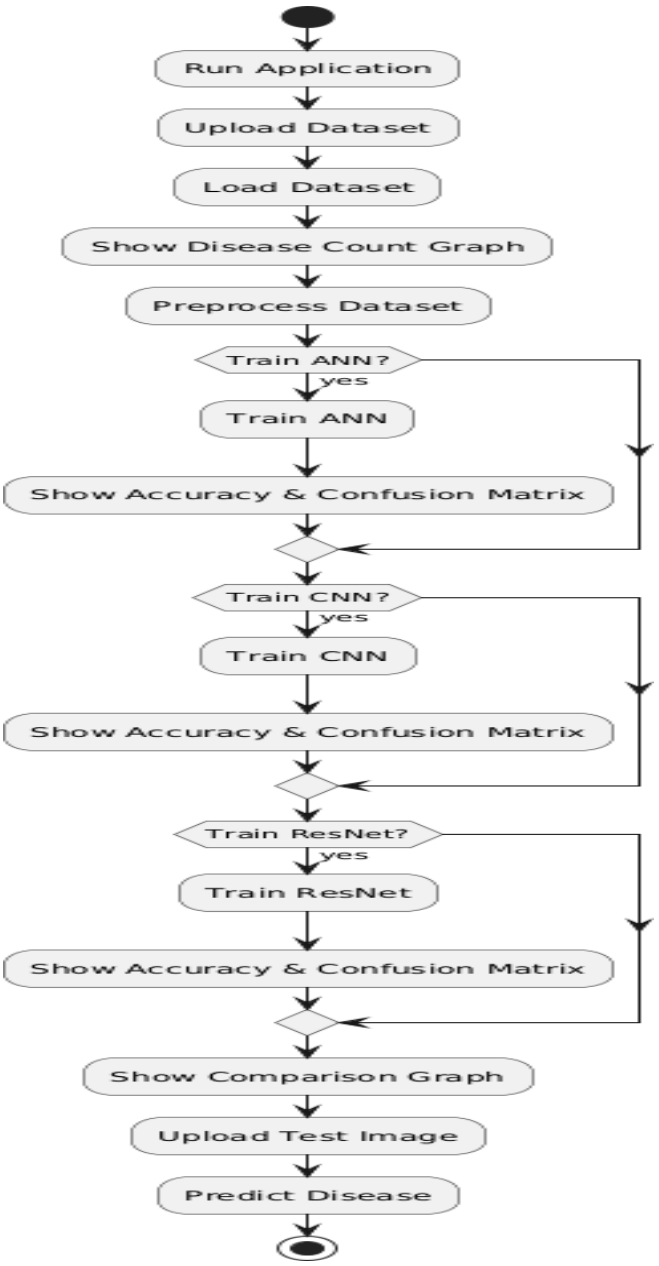


FIGURE 5.3.4 FLOWCHART DIAGRAM

6.IMPLEMENTATION

5. IMPLEMENTATION

5.1 MODULES

USER MODULE

The User Module serves as the primary interface between the end-users (such as farmers, agricultural experts, or extension officers) and the disease recognition system. Its main functions are:

- Image Input: Allows users to upload or capture images of paddy leaves directly through a simple and intuitive interface.
- Result Viewing: Displays the predicted disease name along with confidence levels and possibly additional information like symptoms or control measures.
- User Guidance: Provides instructions on how to take clear images for better accuracy.
- Feedback & Support: Optionally collects user feedback or queries for system improvement or expert consultation.

The User Module is designed to be user-friendly and accessible, requiring no technical expertise, making it suitable for farmers with basic smartphone skills. This module ensures smooth interaction with the backend deep learning system and facilitates timely, actionable information delivery to the users.

6.2 SOURCE CODE

```
from tkinter import messagebox
from tkinter import *
from tkinter import simpledialog
import tkinter
from tkinter import filedialog
import matplotlib.pyplot as plt
import numpy as np
from tkinter.filedialog import askopenfilename
import os
import cv2
from sklearn.metrics import f1_score
import seaborn as sns
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
import pandas as pd
from keras.utils.np_utils import to_categorical
from keras.layers import  MaxPooling2D
from keras.layers import Dense, Dropout, Activation, Flatten
from keras.layers import Convolution2D
from keras.models import Sequential, load_model
import pickle
from sklearn.model_selection import train_test_split
from keras.applications import ResNet101
from sklearn.metrics import accuracy_score
from keras.callbacks import ModelCheckpoint
from keras.layers import Conv2D, MaxPool2D, InputLayer, BatchNormalization, Dropout
main = tkinter.Tk()
main.title("Paddy Crop Disease Detection using Ann, CNN & Resnet101")
main.geometry("1300x1200")
global X, Y, filename, X_train, X_test, y_train, y_test, resnet_model
labels = ['Blast', 'BrownSpot', 'Hispa', 'Normal']
global accuracy, precision, recall, fscore
def getID(name):
    index = 0
    for i in range(len(labels)):
        if labels[i] == name:
            index = i
            break
    return index
def uploadDataset():
    global filename
    filename = filedialog.askdirectory(initialdir=".")
```

```
    sourcelabel.config(text=filename)
    text.delete('1.0', END)
    text.insert(END,filename+" dataset loaded\n\n")
    Y = np.load('model/Y.txt.npy')
    unique, count = np.unique(Y, return_counts = True)
    height = count
    bars = ('Leaf Blast','Brown Spot','Hispa', 'Healthy')
    y_pos = np.arange(len(bars))
    plt.bar(y_pos, height)
    plt.xticks(y_pos, bars)
    plt.xlabel("Disease Names")
    plt.ylabel("Images Count")
    plt.title("Different Diseases found in Dataset")
    plt.show()
def processDataset():
    global X, Y, filename, X_train, X_test, y_train, y_test
    text.delete('1.0', END)
    if os.path.exists("model/X.txt.npy"):
        X = np.load('model/X.txt.npy')
        Y = np.load('model/Y.txt.npy')
    else:
        X = []
        Y = []
        for root, dirs, directory in os.walk(filename):
            for j in range(len(directory)):
                name = os.path.basename(root)
                if 'Thumbs.db' not in directory[j]:
                    img = cv2.imread(root+"/"+directory[j])
                    img = cv2.resize(img, (32, 32))
                    im2arr = np.array(img)
                    im2arr = im2arr.reshape(32, 32, 3)
                    X.append(im2arr)
                    label = getID(name)
                    Y.append(label)
        X = np.asarray(X)
        Y = np.asarray(Y)
        np.save('model/X.txt',X)
        np.save('model/Y.txt',Y)
    X = X.astype('float32')
    X = X/255
    indices = np.arange(X.shape[0])
    np.random.shuffle(indices)
    X = X[indices]
    Y = Y[indices]
    Y = to_categorical(Y)
    X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2) #split dataset into train and
test
```

```
    text.insert(END,"Total Images found in dataset : "+str(X.shape[0])+"\n\n")
    text.insert(END,"Dataset Train & Test Split Details\n\n")
    text.insert(END,"80% dataset used for training : "+str(X_train.shape[0])+"\n")
    text.insert(END,"20% dataset used for training : "+str(X_test.shape[0])+"\n")
    text.update_idletasks()
    test = X[3]
    cv2.imshow("Sample Processed Image",cv2.resize(test,(150,250)))
    cv2.waitKey(0)
#function to calculate all metrics
def calculateMetrics(algorithm, testY, predict):
    p = precision_score(testY, predict,average='macro') * 100
    r = recall_score(testY, predict,average='macro') * 100
    f = f1_score(testY, predict,average='macro') * 100
    a = accuracy_score(testY,predict)*100
    accuracy.append(a)
    precision.append(p)
    recall.append(r)
    fscore.append(f)
    text.insert(END,algorithm+" Accuracy  : "+str(a)+"\n")
    text.insert(END,algorithm+" Precision : "+str(p)+"\n")
    text.insert(END,algorithm+" Recall   : "+str(r)+"\n")
    text.insert(END,algorithm+" FSCORE   : "+str(f)+"\n\n")
    labels = ['Leaf Blast', 'BrownSpot', 'Hispa', 'Healthy']
    conf_matrix = confusion_matrix(testY, predict)
    ax = sns.heatmap(conf_matrix, xticklabels = labels, yticklabels = labels, annot = True,
cmap="viridis" ,fmt ="g");
    ax.set_ylim([0,len(labels)])
    plt.title(algorithm+" Confusion matrix")
    plt.ylabel('True class')
    plt.xlabel('Predicted class')
    plt.show()
def runANN():
    global X_train, X_test, y_train, y_test
    global accuracy, precision, recall, fscore
    text.delete('1.0', END)
    accuracy = []
    precision = []
    recall = []
    fscore = []
    X_train1 = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], (X_train.shape[2] *
X_train.shape[3])))
    X_test1 = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], (X_test.shape[2] *
X_test.shape[3])))
    ann_model = Sequential()
    ann_model.add(Flatten(input_shape=[X_train1.shape[1], X_train1.shape[2]]))
    ann_model.add(Dense(30, activation="relu"))
    ann_model.add(Dense(10, activation="relu"))
```

```
    ann_model.add(Dense(y_train.shape[1],  activation="softmax"))
    ann_model.compile(optimizer = 'adam', loss = 'categorical_crossentropy', metrics = ['accuracy'])
    if os.path.exists("model/ann_weights.hdf5") == False:
        model_check_point = ModelCheckpoint(filepath='model/ann_weights.hdf5', verbose = 1,
save_best_only = True)
        hist = ann_model.fit(X_train1, y_train, batch_size = 16, epochs = 80,
validation_data=(X_test1, y_test), callbacks=[model_check_point], verbose=1)
    else:
       ann_model.load_weights("model/ann_weights.hdf5")
    predict = ann_model.predict(X_test1)
    predict = np.argmax(predict, axis=1)
    y_test1 = np.argmax(y_test, axis=1)
    calculateMetrics("ANN Algorithm", y_test1, predict)
def runCNN():
    global X_train, X_test, y_train, y_test
    global accuracy, precision, recall, fscore
    cnn_model = Sequential()
    cnn_model.add(InputLayer(input_shape=(X_train.shape[1], X_train.shape[2],
X_train.shape[3])))
    cnn_model.add(Conv2D(25, (5, 5), activation='relu', strides=(1, 1), padding='same'))
    cnn_model.add(MaxPool2D(pool_size=(2, 2), padding='same'))
    cnn_model.add(Conv2D(50, (5, 5), activation='relu', strides=(2, 2), padding='same'))
    cnn_model.add(MaxPool2D(pool_size=(2, 2), padding='same'))
    cnn_model.add(BatchNormalization())
    cnn_model.add(Conv2D(70, (3, 3), activation='relu', strides=(2, 2), padding='same'))
    cnn_model.add(MaxPool2D(pool_size=(2, 2), padding='valid'))
    cnn_model.add(BatchNormalization())
    cnn_model.add(Flatten())
    cnn_model.add(Dense(units=100, activation='relu'))
    cnn_model.add(Dense(units=100, activation='relu'))
    cnn_model.add(Dropout(0.25))
    cnn_model.add(Dense(units=y_train.shape[1], activation='softmax'))
    cnn_model.compile(loss='categorical_crossentropy', optimizer="adam", metrics=['accuracy'])
    if os.path.exists("model/cnn_weights.hdf5") == False:
        model_check_point = ModelCheckpoint(filepath='model/cnn_weights.hdf5', verbose = 1,
save_best_only = True)
        hist = cnn_model.fit(X_train, y_train, batch_size = 16, epochs = 80, validation_data=(X_test,
y_test), callbacks=[model_check_point], verbose=1)
    else:
        cnn_model.load_weights("model/cnn_weights.hdf5")
    predict = cnn_model.predict(X_test)
    predict = np.argmax(predict, axis=1)
    y_test1 = np.argmax(y_test, axis=1)
    calculateMetrics("CNN Algorithm", y_test1, predict)
def runResnet():
    global X_train, X_test, y_train, y_test
    global accuracy, precision, recall, fscore, resnet_model
```

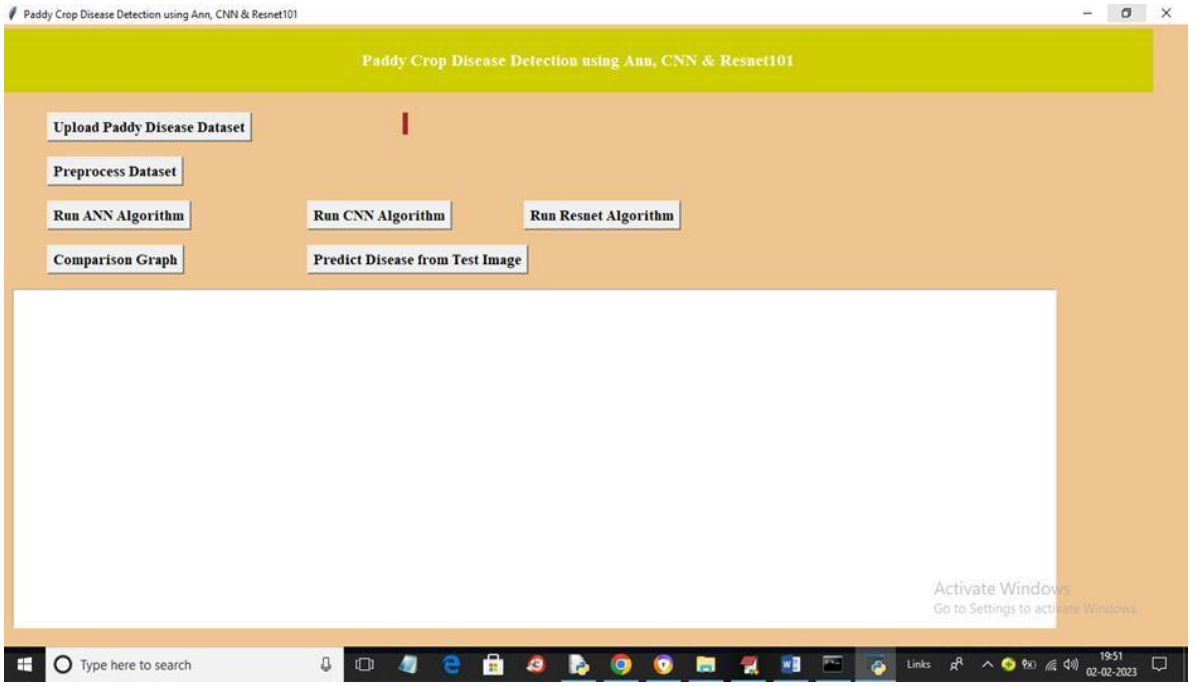
```
    resnet = ResNet101(include_top=False, weights='imagenet', input_shape=(X_train.shape[1],
X_train.shape[2], X_train.shape[3]))
    for layer in resnet.layers:
        layer.trainable = False
    resnet_model = Sequential()
    resnet_model.add(resnet)
    resnet_model.add(Convolution2D(32, (1, 1), input_shape = (X_train.shape[1], X_train.shape[2],
X_train.shape[3]), activation = 'relu'))
    resnet_model.add(MaxPooling2D(pool_size = (1, 1)))
    resnet_model.add(Convolution2D(32, (1, 1), activation = 'relu'))
    resnet_model.add(MaxPooling2D(pool_size = (1, 1)))
    resnet_model.add(Flatten())
    resnet_model.add(Dense(units = 256, activation = 'relu'))
    resnet_model.add(Dense(units = y_train.shape[1], activation = 'softmax'))
    print(resnet_model.summary())
    resnet_model.compile(optimizer = 'adam', loss = 'categorical_crossentropy', metrics =
['accuracy'])
    if os.path.exists("model/resnet_weights.hdf5") == False:
        model_check_point = ModelCheckpoint(filepath='model/resnet_weights.hdf5', verbose = 1,
save_best_only = True)
        hist = resnet_model.fit(X_train, y_train, batch_size = 16, epochs = 80,
validation_data=(X_test, y_test), callbacks=[model_check_point], verbose=1)
    else:
        resnet_model = load_model("model/resnet_weights.hdf5")
    predict = resnet_model.predict(X_test)
    predict = np.argmax(predict, axis=1)
    y_test1 = np.argmax(y_test, axis=1)
    calculateMetrics("ResNet Algorithm", y_test1, predict)
def graph():
    df = pd.DataFrame([['ANN','Precision',precision[0]],['ANN','Recall',recall[0]],['ANN','F1
Score',fscore[0]],['ANN','Accuracy',accuracy[0]],
                    ['CNN','Precision',precision[1]],['CNN','Recall',recall[1]],['CNN','F1
Score',fscore[1]],['CNN','Accuracy',accuracy[1]],
                    ['ResNet','Precision',precision[2]],['ResNet','Recall',recall[2]],['ResNet','F1
Score',fscore[2]],['ResNet','Accuracy',accuracy[2]],
                ],columns=['Parameters','Algorithms','Value'])
    df.pivot("Parameters", "Algorithms", "Value").plot(kind='bar')
    plt.show()
def predict():
    global resnet_model
    labels = ['Leaf Blast', 'BrownSpot', 'Hispa', 'Healthy']
    text.delete('1.0', END)
    filename = filedialog.askopenfilename(initialdir="testImages")
    image = cv2.imread(filename)
    img = cv2.resize(image, (32, 32))
    im2arr = np.array(img)
    im2arr = im2arr.reshape(1,32,32,3)
```

```
    img = np.asarray(im2arr)
    img = img.astype('float32')
    img = img/255
    preds = resnet_model.predict(img)
    predict = np.argmax(preds)
    max_value = np.amax(preds)
    if max_value > 0.95:
        img = cv2.imread(filename)
        img = cv2.resize(img, (700,400))
        cv2.putText(img, 'Paddy Disease Predicted as : '+labels[predict], (10, 25),
cv2.FONT_HERSHEY_SIMPLEX,0.7, (255, 0, 0), 2)
        cv2.imshow('Paddy Disease Predicted as : '+labels[predict], img)
        cv2.waitKey(0)
    else:
        img = cv2.imread(filename)
        img = cv2.resize(img, (700,400))
        cv2.putText(img, 'Not a paddy leaf image', (10, 25),  cv2.FONT_HERSHEY_SIMPLEX,0.7,
(255, 0, 0), 2)
        cv2.imshow('Not a paddy leaf image', img)
        cv2.waitKey(0)
font = ('times', 14, 'bold')
title = Label(main, text='Paddy Crop Disease Detection using Ann, CNN & Resnet101')
title.config(bg='yellow3', fg='white')
title.config(font=font)
title.config(height=3, width=120)
title.place(x=0,y=5)
font1 = ('times', 13, 'bold')
uploadButton = Button(main, text="Upload Paddy Disease Dataset", command=uploadDataset)
uploadButton.place(x=50,y=100)
uploadButton.config(font=font1)
sourcelabel = Label(main)
sourcelabel.config(bg='brown', fg='white')
sourcelabel.config(font=font1)
sourcelabel.place(x=460,y=100)
processButton = Button(main, text="Preprocess Dataset", command=processDataset)
processButton.place(x=50,y=150)
processButton.config(font=font1)
annButton = Button(main, text="Run ANN Algorithm", command=runANN)
annButton.place(x=50,y=200)
annButton.config(font=font1)
cnnButton = Button(main, text="Run CNN Algorithm", command=runCNN)
cnnButton.place(x=350,y=200)
cnnButton.config(font=font1)
resnetbutton = Button(main, text="Run Resnet Algorithm", command=runResnet)
resnetbutton.place(x=600,y=200)
resnetbutton.config(font=font1)
graphbutton = Button(main, text="Comparison Graph", command=graph)
```

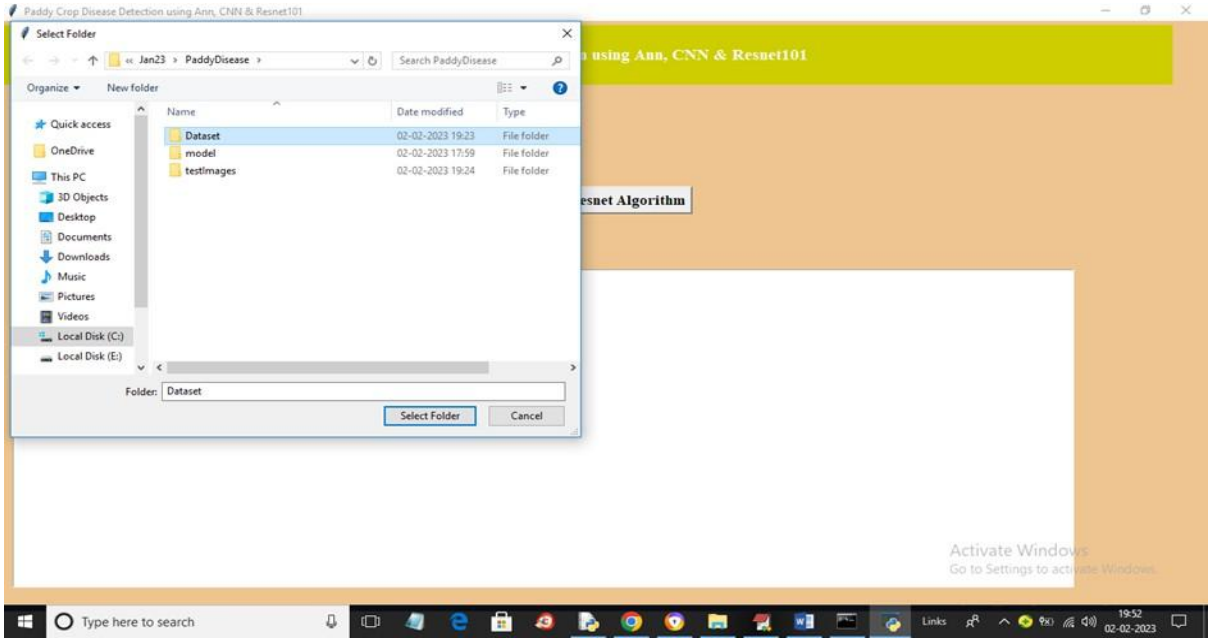
```
graphbutton.place(x=50,y=250)
graphbutton.config(font=font1)
predictbutton = Button(main, text="Predict Disease from Test Image", command=predict)
predictbutton.place(x=350,y=250)
predictbutton.config(font=font1)
font1 = ('times', 12, 'bold')
text=Text(main,height=20,width=150)
scroll=Scrollbar(text)
text.configure(yscrollcommand=scroll.set)
text.place(x=10,y=300)
text.config(font=font1)
main.config(bg='burlywood2')
main.mainloop()
```

7.SCREENSHOTS

6. SCREENSHOTS

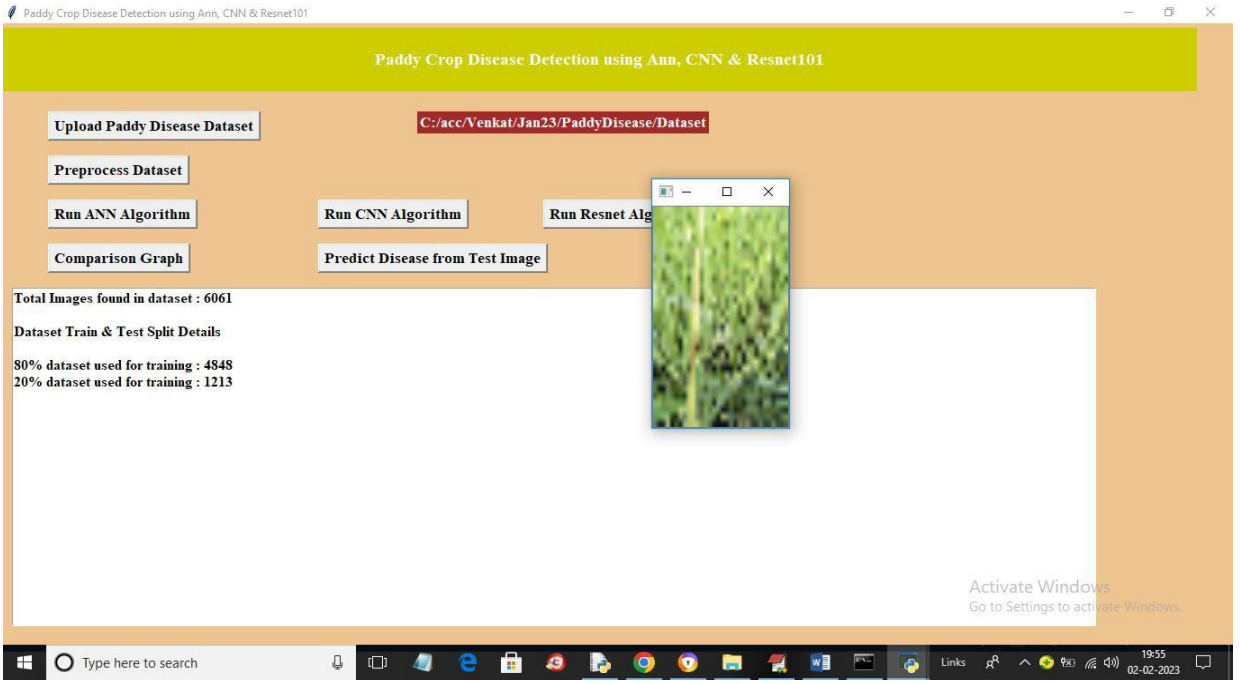


Screenshot 7.1 : Welcome Screen of Image-based Paddy Disease Recognition using ResNet101 Deep Network



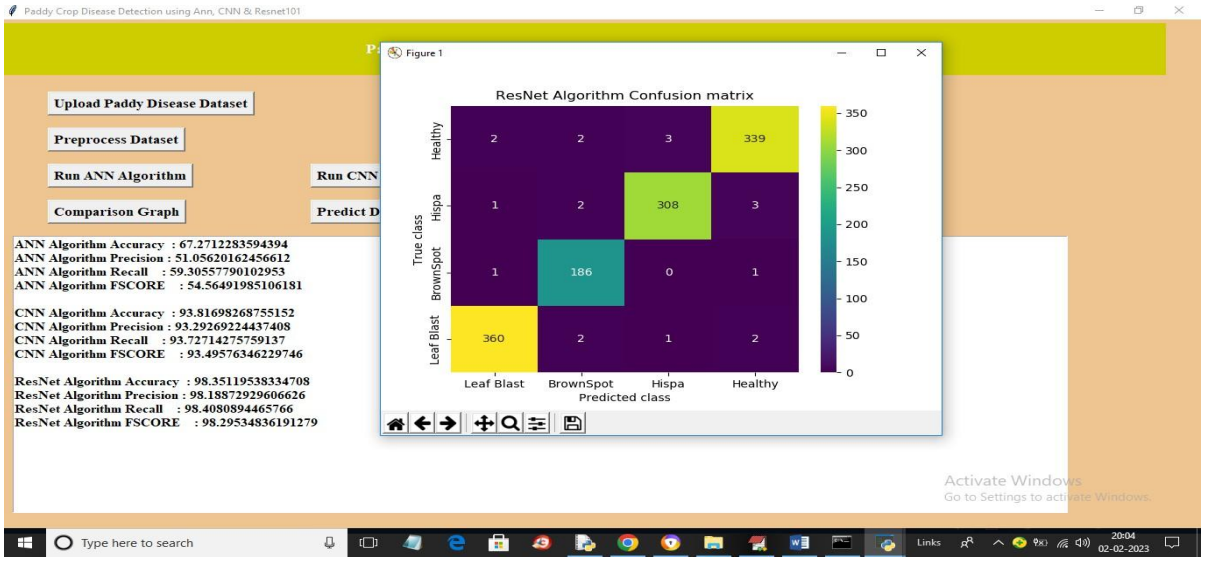
Screenshot 7.2 : Uploading Dataset

In above screen selecting and uploading entire dataset folder and then click on 'Dataset' button to load dataset and get below output.



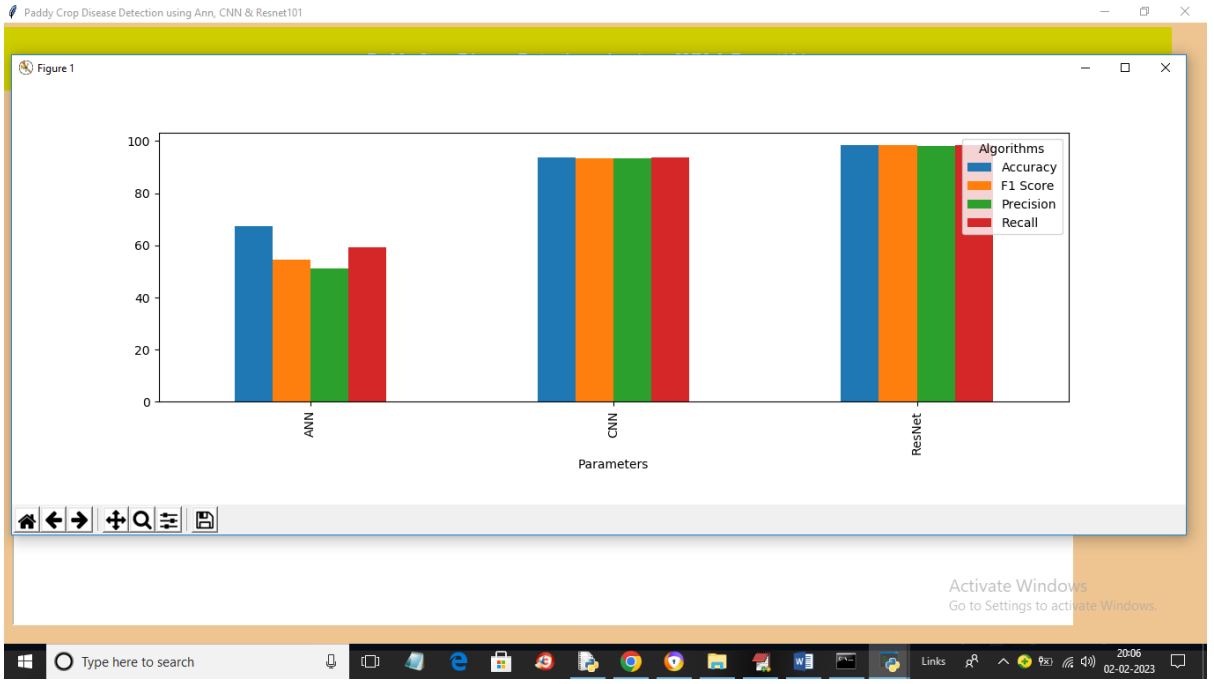
Screenshot 7.3 : Preprocessing the Dataset

In above screen we can see dataset contains 6061 images and application using 80% (4848) images for training and 20% (1213) images for testing and we can see sample processed image also and now close that image and dataset is ready and now click on 'Run ANN Algorithm' button to train ANN and get below output.



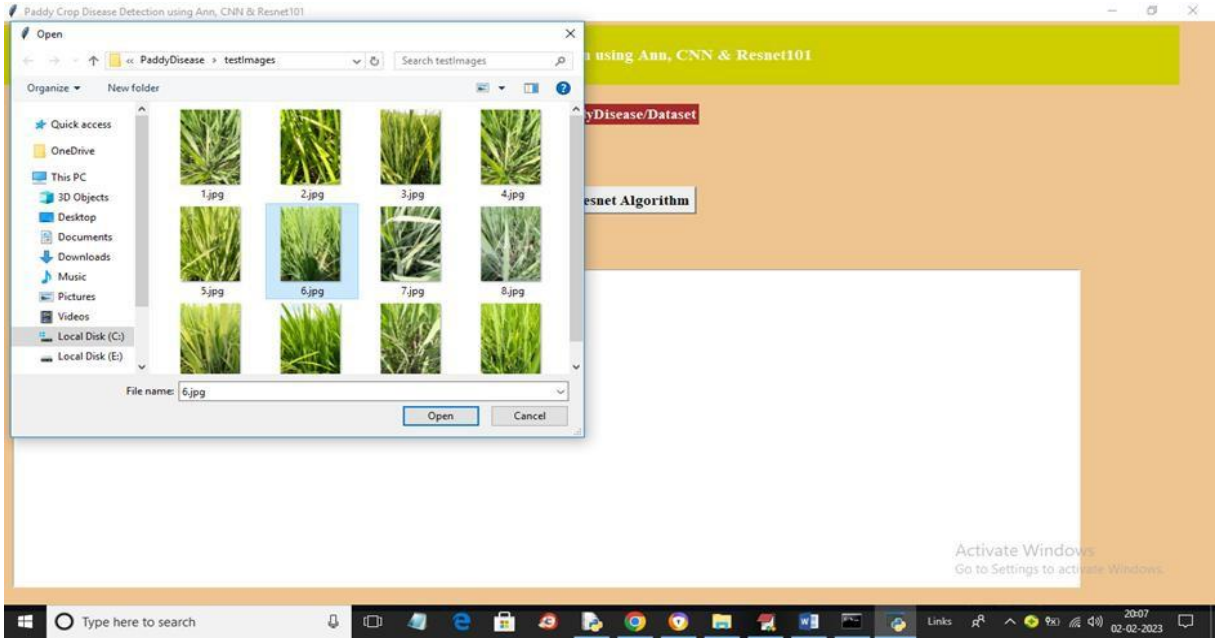
Screenshot 7.4 : Running ResNet Algorithm

In above screen with Resnet we got 98% accuracy and in blue boxes only 2 or 3 images are incorrectly predicted and in diagonal different colour boxes we can see more number of images are predicted.

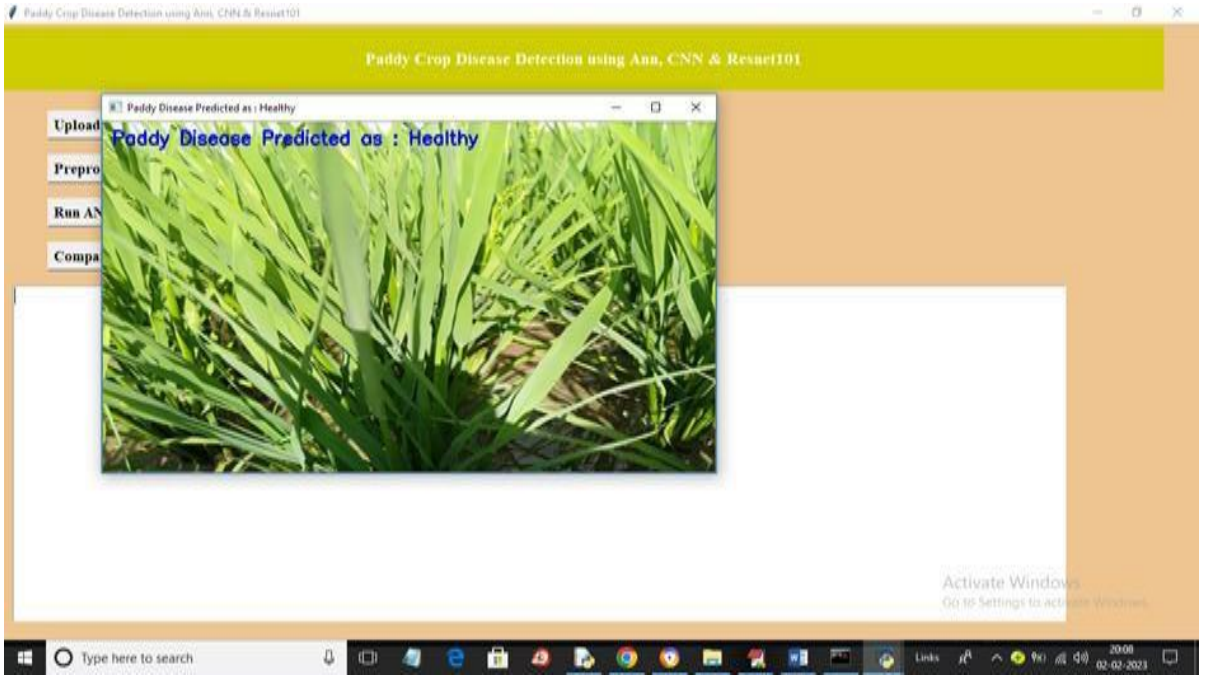


Screenshot 7.5 : Comparison Graph

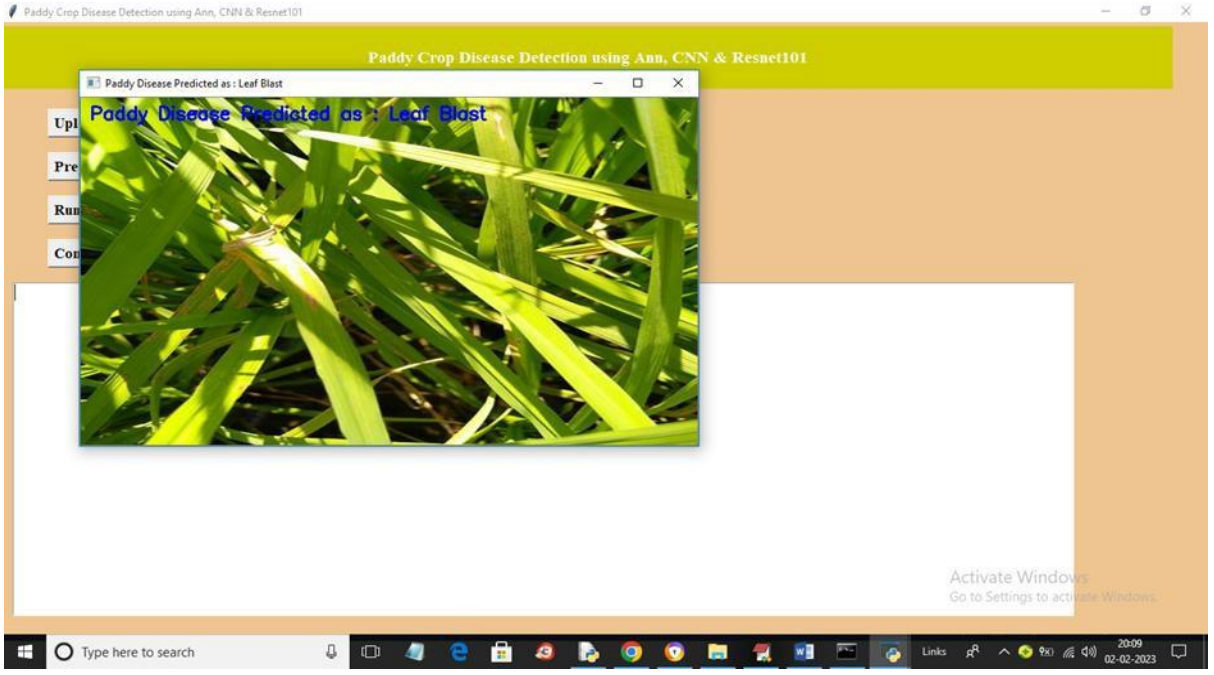
In above graph x-axis represents algorithm names and y-axis represents accuracy and other metrics in different colour bars and in all algorithms RESNET got high accuracy.



Screenshot 7.6 : Predict Disease from Test Image



Screenshot 7.7 : Predicting the Disease as Healthy



Screenshot 7.8 : Predicting Disease as Leaf Blast

8. TESTING

8. TESTING

8.1 INTRODUCTION TO TESTING

Testing is a crucial process aimed at discovering errors, faults, or weaknesses in a software product. It involves systematically checking the functionality of components, subassemblies, assemblies, or the final product to ensure that it meets its intended requirements and user expectations. The primary objective of testing is to validate that the software functions correctly under different conditions and does not fail in unacceptable ways. Various types of testing exist, each designed to address specific aspects or requirements of the software, ensuring its overall quality and reliability.

8.2 TYPES OF TESTING

8.2.1 UNIT TESTING

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit test ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

8.2.2 INTEGRATION TESTING

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent.

8.2.3 FUNCTIONAL TESTING

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current test is determined.

8.2.4 SYSTEM TESTING

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

8.2.5 WHITEBOX TESTING

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or atleast its purpose. It is used to test area that Can not be reached from a black box level.

8.2.6 BLACKBOX TESTING

Black Box Testing is a testing method where the software is tested without any knowledge of its internal structure, code, or implementation. Testers rely on source documents, like specifications or requirements, to provide inputs and evaluate outputs, focusing only on the functionality rather than the internal workings of the software.

8.2.7 PERFORMANCE TESTING

Performance testing is a type of software testing that focuses on evaluating the performance and scalability of a system or application. Performance testing aims to identify bottlenecks, measure system performance under various loads and conditions, and ensure that the system can handle the expected number of users or transactions.

8.2.8 ACCEPTANCE TESTING

UserAcceptanceTesting is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

8.3 TESTCASES

TestCase ID	Test Case Description	Expected Result	Actual Result	Status
TC-001	Upload paddy leaf image	Image uploaded successfully	Image uploaded successfully	Passed
TC-002	Predict disease from valid paddy image	Disease predicted (e.g., "Brown Spot")	Disease predicted correctly	Passed
TC-003	Upload non-paddy image	Error message displayed	Error message displayed	Passed
TC-004	Predict disease with healthy leaf image	Prediction result: "Healthy"	Prediction result: "Healthy"	Passed
TC-005	Display prediction with confidence score	Prediction with accuracy score (e.g., 89.5%)	Accuracy score shown	Passed

9. CONCLUSION
&
FUTURE SCOPE

9.CONCLUSION & FUTURE SCOPE

9.1 PROJECT CONCLUSION

The project titled "Image-Based Paddy Disease Recognition Using ResNet-101" presents an effective and intelligent approach for identifying diseases in paddy leaves through image classification. By utilizing the deep learning architecture ResNet-101, the system achieves high accuracy in detecting and classifying multiple paddy diseases such as bacterial blight, blast, brown spot, and leaf smut.The use of ResNet-101 enables the extraction of deep and meaningful features from leaf images, overcoming challenges such as variations in lighting, background noise, and similarity between disease patterns. The model was trained and evaluated on a dataset of paddy leaf images, showing reliable performance in real-world conditions.This project demonstrates how advanced computer vision techniques can be applied in the agricultural sector to support early diagnosis and disease management. The proposed system can be integrated into mobile applications or web platforms, enabling farmers to identify crop diseases in real-time without the need for expert knowledge.In summary, the project contributes to the development of smart and sustainable agriculture by providing a cost-effective, automated, and accurate solution for paddy disease detection. It holds significant potential to reduce crop loss, improve yield, and support farmers with timely information for better crop management.

9.2 FUTURE SCOPE

The project "Image-Based Paddy Disease Recognition Using ResNet-101" has demonstrated the effectiveness of deep learning in crop disease detection; however, it offers ample opportunities for future enhancement. One major direction is expanding the system to support multiple crops and a broader range of diseases, making it more versatile across various agricultural domains. The integration of the model into a real-time mobile application would significantly benefit farmers by enabling instant disease detection through smartphone cameras, even in rural and low-connectivity areas. Future work can also focus on improving the model’s accuracy by incorporating a larger and more diverse dataset, collected under different environmental and lighting conditions. Features such as geolocation tagging, weather-based risk prediction, and integration with agricultural advisory systems can further increase the model’s usability, helping farmers make better and timely decisions for crop management.

10.BIBLIOGRAPHY

10 BIBLIOGRAPHY

[1] S. B. Thamarai Selvi *et al.*, "Deep learning based paddy disease classification using ResNet-50,"*Proc. IACIDS*, 2024. ResNet-based CNN achieves high accuracy for paddy disease detection.

[2] O. Attallah, "RiPa-Net: Recognition of rice paddy diseases using CNN," *Biomimetics*, vol. 8, no. 5, 2023.
Proposes a dual-layer CNN model for accurate rice disease recognition.

[3] P. Pai *et al.*, "Deep learning-based automatic diagnosis of rice leaf diseases," *Scientific Reports*, 2025.
Uses ensemble CNN models for robust and precise disease classification.

[4] P. Seelwal *et al.*, "Deep learning applications for rice disease diagnosis: A review," *Frontiers in Computer Science*, 2024.
Reviews modern deep learning techniques for rice disease detection.

[5] Y. Li *et al.*, "Deep learning-based multi-class rice disease detection," *Agronomy*, vol. 14, no. 9, 2024.
Focuses on multi-class classification of rice diseases using CNN models.

[6] M. Gowthami *et al.*, "Paddy leaves disease categorization using ResNet in CNN," *Proc. ICICC*, 2023.
Applies ResNet architecture for efficient paddy disease classification.

[7] Artificial Intelligence in Agriculture, "Comparison of CNN architectures for rice disease classification," vol. 9, pp. 22–35, 2023.
Compares multiple CNN models achieving high classification accuracy.

[8] P. A. S. Rani and N. S. Singh, "Paddy leaf disease classification using ResNet-50," *IJASCE*, vol. 4, no. 2, 2022.
Demonstrates transfer learning using ResNet for disease prediction.

11.GITHUB LINK

Github Link :

https://github.com/Rajeshreddy264/IMAGE-BASED-PADDY-DISEASE-RECOGNITION-USING-RESNET101-DEEP-NETWORK.git